

UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO - COORDINACIÓN DE INFORMÁTICA
CÁTEDRA: INGENIERÍA DE SOFTWARE II

Profesor: Mg. Félix Márquez | Estudiante: Brayan (Integrante 4)
Unidad V: Gestión de Configuración, Versionamiento y DevOps
Septiembre de 2026

RESOLUCIÓN TÉCNICA: EJERCICIO 2 Y PREGUNTA 4

Tópico de Especialidad: Everything as Code, Contenedorización Inmutable, Paridad de Entornos y Auditoría Moderna de SCM

1. EJERCICIO 2: EVERYTHING AS CODE Y GESTIÓN DE ENTORNOS

1.1. Diagnóstico del Problema Operativo y Deriva de Configuración

En el contexto analizado, la organización sufre severamente las repercusiones del síndrome conocido en la industria como «en mi máquina sí funciona». Dicha patología técnica surge a raíz de la ausencia de una línea base formal para los entornos de desarrollo y producción. Cada ingeniero configura manualmente su estación de trabajo con dependencias globales dispersas, versiones no alineadas del runtime de Node.js y parches heterogéneos en el sistema operativo anfitrión. Paralelamente, el departamento de operaciones aprovisiona los servidores mediante scripts de Bash imperativos, no versionados y no idempotentes.

Este desacoplamiento ocasiona la temida deriva de configuración (Configuration Drift): una divergencia no controlada e invisible entre entornos donde los servidores se convierten en «servidores copo de nieve» (snowflake servers). Como consecuencia, artefactos que aprueban los ensayos locales colapsan catastróficamente durante la puesta en producción debido a incompatibilidades de librerías nativas, permisos de socket o variables de entorno no documentadas.

1.2. Dockerización de la Aplicación Node.js (Puerto 8080)

Para erradicar la variabilidad del entorno y garantizar una entrega determinista, se define la infraestructura como código ejecutable mediante el siguiente Dockerfile, optimizado conforme a los estándares de la SCM moderna y la disciplina DevSecOps:

```
# 1. IMAGEN BASE INMUTABLE:
FROM node:20.17.0-alpine3.20

# 2. METADATOS Y VARIABLES DE ENTORNO:
ENV NODE_ENV=production \
    PORT=8080

# 3. DIRECTORIO DE TRABAJO AISLADO:
WORKDIR /usr/src/app

# 4. OPTIMIZACIÓN DE LA CACHE DE CAPAS (BUILD CACHING):
COPY package*.json ./

# 5. INSTALACIÓN LIMPIA Y DETERMINISTA:
RUN npm ci --only=production && \
    npm cache clean --force

# 6. TRANSFERENCIA DEL CÓDIGO CON PROPIEDAD DE USUARIO:
COPY --chown=node:node . .

# 7. PRINCIPIO DE MENOR PRIVILEGIO (SEGURIDAD DEVSECOPS):
USER node

# 8. DECLARACIÓN FORMAL DE PUERTO:
EXPOSE 8080

# 9. COMPROBACIÓN DE SALUD ACTIVA (HEALTHCHECK):
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
    CMD wget --no-verbose --tries=1 --spider http://localhost:8080/health || exit 1

# 10. INVOCACIÓN INMUTABLE DE ENTRADA (SINTAXIS EXEC):
CMD ["node", "server.js"]
```

1.3. Mecanismo de Eliminación de la Deriva de Configuración

El archivo Dockerfile resuelve de raíz la deriva de configuración apoyándose en tres principios arquitectónicos de la ingeniería de software moderna:

1. Inmutabilidad del Entorno de Ejecución (Immutable Infrastructure): En la administración tradicional de servidores, los entornos eran tratados como entidades mutables («mascotas») sobre las cuales se aplicaban parches en caliente. Con Docker, el entorno se concibe como infraestructura descartable («ganado»). La imagen generada es un objeto binario de sólo lectura, sellado criptográficamente por capas mediante hashes SHA-256. Todo cambio en el runtime exige la construcción de una nueva imagen versionada, eliminando modificaciones manuales en caliente.
2. Encapsulamiento Holístico y Autosuficiente: A diferencia de los scripts de Bash que dependen del estado previo del host, el contenedor encapsula absolutamente todas las dependencias requeridas para la ejecución: la versión exacta del kernel compatible, la distribución mínima de Alpine Linux (3.20), el intérprete de Node.js (v20.17.0), las librerías dinámicas del sistema, los manifiestos de NPM y el código fuente. La aplicación ya no interactúa con el sistema operativo host de forma acoplada.
3. Paridad Estricta entre Desarrollo y Producción (Dev/Prod Parity): Conforme al décimo principio del manifiesto The Twelve-Factor App, la distancia entre el ambiente local del programador y el cluster de producción se reduce a cero. El mismo paquete binario que el desarrollador prueba en su máquina es idéntico bit por bit al que se ejecuta en el servidor de pruebas y en el entorno productivo.

1.4. Reflexión SCM: Ventajas de Trazabilidad y Auditoría de IaC en Git vs. Logs Manuales

Bajo los marcos de referencia de la SCM clásica (IEEE Std 828 e IEEE SWEBOK V4), la integridad de un sistema se fundamenta en la Trazabilidad (capacidad de asociar cualquier cambio a un requerimiento y a una versión formal) y en la Auditoría de Configuración (verificación independiente de que el producto desplegado cumple con su especificación formal).

Al comparar la Infraestructura como Código (IaC) gobernada por Git frente a los logs manuales y scripts Bash no versionados de un SysAdmin, emergen diferencias concluyentes:

Dimensión SCM	Administración Tradicional (Bash y Logs Manuales)	Infraestructura como Código en Git (IaC)
Definición de Línea Base	Ambigua y mutable. La línea base es el estado acumulado en el disco del servidor tras meses de comandos manuales no documentados.	Declarativa y unívoca. La línea base es el hash del commit en Git. Cualquier modificación queda registrada en la historia formal.
Trazabilidad de Cambios	Fragmentada y vulnerable. Los comandos quedan en bitácoras manuales o .bash_history, susceptibles de alteración o borrado accidental.	Criptográfica e inmutable. Se conoce con certeza quién modificó el archivo (Author), cuándo (Timestamp) y por qué motivo (Commit Message y Pull Request).
Aprobación del Cambio	Burocrática y a posteriori. Se ejecuta el cambio en caliente y luego se elabora el informe, imposibilitando la validación preventiva.	Preventiva y automatizada. Los cambios en IaC se someten a revisión por pares (Peer Review) y a pruebas automatizadas en CI antes del merge.
Idempotencia	No idempotente. Reejecutar un script de Bash sobre un servidor previamente alterado genera efectos colaterales y colisiones.	Totalmente idempotente. El manifiesto declarativo especifica el estado final requerido; el motor de contenedores garantiza resultados idénticos.
Recuperación ante Desastres (MTTR)	Manual y prolongada. Reconstruir un nodo exige reinstalar dependencias paso a paso, aumentando drásticamente el tiempo de caída.	Inmediata y automatizada. Un nuevo clúster se aprovisiona en minutos ejecutando el pipeline de CI/CD desde el repositorio Git (GitOps).
Auditoría Funcional y Física	Inviabile formalmente. No hay mecanismo objetivo para certificar qué parches o binarios residen en el sistema de archivos.	Continua y verificable. Se audita en tiempo real comparando la firma criptográfica del contenedor en producción con el hash en Git.

2. PREGUNTA 4: ARTEFACTOS MODERNOS Y AUDITORÍA EN SCM

2.1. La Evolución Paradigmática: Del Plan en Word a los Manifiestos de Repositorio

En las etapas formativas de la Ingeniería de Software, la gestión de configuración descansaba sobre el «Plan de Gestión de Configuraciones del Software» (SCMP, por sus siglas en inglés), formalizado bajo normativas como IEEE Std 828. Este artefacto solía ser un documento estático de procesador de texto (Microsoft Word) de decenas o cientos de páginas que describía

exhaustivamente las convenciones de nomenclatura, las herramientas de construcción permitidas, el organigrama de la Junta de Control de Cambios (CCB) y los procedimientos manuales de auditoría.

No obstante, dicho modelo presentaba una debilidad operativa insalvable: el divorcio absoluto entre el documento y la realidad del código fuente. Por ser un artefacto redactado en lenguaje natural pasivo, envejecía irremediabilmente a las pocas semanas de iniciado el proyecto. La disciplina DevOps y el enfoque Everything as Code (EaC) superaron esta brecha reemplazando la prosa burocrática por especificaciones declarativas legibles por máquina que residen en la raíz del mismo repositorio.

2.2. Identificación y Análisis de los Cuatro Artefactos Clave de SCM

En la actualidad, la integridad y el control de un Ítem de Configuración de Software (SCI) se preservan mediante los siguientes cuatro archivos declarativos:

1. Manifiesto .gitignore: El Filtro de Integridad de la Línea Base

El archivo `.gitignore` es la primera línea de defensa de la SCM moderna. Su propósito es definir patrones de exclusión de manera que archivos volátiles, binarios generados o información confidencial no sean indexados en el área de preparación (staging area) ni queden registrados en el historial de Git.

- **Integridad de la Línea Base:** Evita que dependencias reproducibles masivas (como la carpeta `node_modules/`) contaminen el repositorio, conservando el grafo de Git liviano y libre de conflictos de mezcla generados por binarios externos.
- **Seguridad y Gobernanza DevSecOps:** Impide la fuga catastrófica de secretos corporativos, tales como claves privadas RSA (`.pem`), certificados SSL o archivos de variables de entorno locales (`.env`), garantizando que las credenciales no penetren en el historial distribuido.

- Neutralidad del Entorno: Excluye artefactos transitorios propios del IDE o del sistema operativo del desarrollador (.DS_Store, carpetas .vscode/), asegurando que las preferencias personales no alteren la línea base colectiva.

2. Manifiesto Dockerfile: La Especificación Inmutable del Entorno de Ejecución

El Dockerfile encapsula la receta declarativa de ensamblaje del entorno de cómputo. En la SCM clásica, auditar si un servidor poseía los parches correctos requería inventarios manuales tediosos. El Dockerfile convierte todo ese estado de infraestructura en un objeto versionado junto al código.

- Identificación Rigurosa del SCI: La imagen compilada a partir del Dockerfile actúa como una unidad sellada identificable por su hash SHA-256 inmutable, permitiendo a los auditores cotejar en milisegundos si la imagen activa en producción corresponde exactamente a la aprobada.
- Reproducibilidad Extrema: Al fijar etiquetas específicas de dependencias y emplear comandos limpios de instalación (npm ci), garantiza que la construcción sea determinista e idéntica sin importar en qué fecha o servidor se ejecute.

3. Pipeline Declarativo .github/workflows/ci.yml: El Control de Cambios Automatizado

Este archivo en formato YAML formaliza la arquitectura del pipeline de Integración Continua (CI). Representa la digitalización y automatización estricta de la antigua Junta de Control de Cambios (CCB).

- Calidad Continua y Quality Gates: Codifica las condiciones innegociables para admitir un cambio en la línea base: ejecución impecable de pruebas unitarias, análisis estático de vulnerabilidades (SAST), validación de estilos de codificación y verificación de construcción del contenedor.

- Trazabilidad y Bloqueo Algorítmico: Git vincula la ejecución del workflow directamente al hash del commit y al Pull Request correspondiente. Si alguna prueba falla, el sistema inhabilita técnicamente la opción de Merge, eliminando la complacencia humana en el control de calidad.

4. Manifiesto docker-compose.yml: La Topología de Configuración Multicapa

Mientras el Dockerfile define un servicio aislado, docker-compose.yml formaliza la arquitectura del sistema distribuido completo (backend, frontend, base de datos relacional y broker de eventos).

- Integridad de la Arquitectura de Red y Servicios: Sustituye los diagramas estáticos de arquitectura por una especificación ejecutable. Formaliza los puertos expuestos, el aislamiento mediante subredes virtuales privadas, el orden riguroso de inicio de microservicios (depends_on con healthchecks) y el mapeo de volúmenes persistentes.

- Reproducibilidad del Ecosistema Completo: Confiere a cualquier miembro del equipo la capacidad de aprovisionar con un único comando un ambiente local de pruebas idéntico al clúster productivo, resolviendo las colisiones de red mediante DNS interno automatizado.

2.3. Matriz de Garantías de SCM: Integridad, Reproducibilidad y Control

La siguiente matriz resume de forma concluyente cómo cada uno de estos cuatro artefactos responde ante los tres requerimientos fundamentales de la disciplina de Gestión de Configuración:

Artefacto Moderno	Garantía de Integridad	Garantía de Reproducibilidad	Garantía de Control
.gitignore	Mantiene la base de código pura, libre de binarios locales volátiles y previene la exposición de credenciales.	Asegura que el repo almacene solo código primario; las dependencias se reconstruyen limpiamente sin artefactos ajenos.	Delimita formalmente las fronteras del Ítem de Configuración que se encuentra bajo el control de versiones de Git.
Dockerfile	Sella el sistema operativo, el runtime y la aplicación en un contenedor inmutable contra alteraciones ad-hoc.	Construcción determinista gracias a tags base fijados y comandos de instalación estricta (npm ci).	Cada cambio de infraestructura requiere un commit auditable en Git con revisión de código antes de su despliegue.

.github/workflows/ci.yml	Aplica barreras de calidad automatizadas (Quality Gates), pruebas y escaneos antes de autorizar la integración.	El proceso de compilación y verificación corre sobre máquinas limpias y homogéneas en cada commit.	Reemplaza la firma de un comité por un árbitro algorítmico verificable que bloquea el merge ante cualquier error.
docker-compose.yml	Formaliza la topología de red, aislamiento de servicios y límites de hardware, blindando la cohesión del sistema.	Todo el ecosistema multicapa se despliega idéntico en cualquier máquina anfitriona mediante una instrucción declarativa.	Centraliza en un archivo versionable las políticas de enlace y las variables de conexión entre microservicios.

3. CONCLUSIONES Y SÍNTESIS ACADÉMICA

La evolución desde el Plan de Gestión de Configuraciones en papel hacia el paradigma Everything as Code (EaC) constituye una de las transformaciones metodológicas y operativas más significativas de la ingeniería moderna de software. Al codificar las políticas de configuración en archivos declarativos dentro de Git, la gobernanza del software abandona la burocracia documental para convertirse en un mecanismo vivo, ejecutable, determinista y auditado de forma continua.

4. REFERENCIAS BIBLIOGRÁFICAS (NORMATIVA APA)

- Bass, L., Clements, P., & Kazman, R. (2021). *Software Architecture in Practice* (4th ed.). Addison-Wesley.
- Driessen, V. (2010). A successful Git branching model. nvie.com.
- Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations*. IT Revolution Press.
- Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.

IEEE Computer Society. (2014). Guide to the Software Engineering Body of Knowledge (SWEBOK V3/V4). IEEE.

Prokic, S. (2021). Infrastructure as Code: Dynamic Infrastructure with Terraform and Docker. Packt Publishing.